

Homework 1

A1) Certainly, there are interdependencies among certain security concepts. Take, for instance, the relationship between authentication and authorization. Authentication frequently serves as a foundational step for authorization. To ascertain a user's eligibility to perform a specific action, the system must initially confirm their identity via authentication. The rationale behind this is that authorization deals with validating a user's permissions, whereas authentication focuses on confirming their identity. Nonetheless, it's important to note that not all security concepts rely on one another, and some can be autonomously implemented.

A2) Dual-factor authentication surpasses single-factor authentication due to its requirement for users to provide two distinct sets of credentials to confirm their identity. This added layer of complexity poses a significant hurdle for potential attackers attempting unauthorized access. In the case of single-factor authentication, like relying solely on a password, an attacker only needs to acquire or guess that single piece of information to gain entry. In contrast, two-factor authentication necessitates the possession of both elements, such as a password along with a physical token or biometric data, rendering unauthorized access considerably more challenging. As a result, it stands as a highly secure authentication method, particularly well-suited for safeguarding sensitive systems and data.

A3) The Bell-LaPadula model operates as a security framework designed to protect confidentiality by meticulously controlling the flow of data across different security levels. In this model, data is categorized into distinct security levels, and access to information is tightly regulated based on a user's security clearance level. Moreover, the model strictly adheres to the "no read up, no write down" principle, which means that a user with a lower security clearance level is prohibited from both accessing and modifying data at a higher security clearance level.

In the described scenario, if an attacker with an unclassified security clearance manages to deceive their boss (who has a top-secret clearance) into running a Trojan horse, the Bell-LaPadula model would prevent the attacker from accessing top-secret information. This is because the attacker's security clearance level is lower than that of the top-secret information. The model would effectively block the attacker from "reading up" to access the top-secret data, even if they gained access to the system through the Trojan horse.

A4) Confidentiality pertains to the preservation of information's secrecy, shielding it from unauthorized access. On the other hand, data integrity concerns the assurance of information accuracy and its protection against unauthorized alterations or tampering. To put it differently, confidentiality focuses on averting unauthorized disclosure, whereas data integrity centers on thwarting unauthorized modifications.

A5) The SimpleWebServer would maintain an open connection and wait indefinitely for the client to transmit data, potentially monopolizing server resources and obstructing other clients' access. This situation aligns with a known attack called a Denial of Service (DoS) attack, wherein an aggressor seeks to disrupt the regular operations of a server or network by inundating it with excessive traffic or connections. In this particular scenario, the attacker could occupy server resources by leaving connections open without transmitting any data, effectively blocking legitimate clients from establishing connections or reaching the server.

A6 a) In the event that SimpleWebServer were equipped with file uploading capabilities, several potential threats would necessitate consideration:

- **Malicious File Uploads:** The system might be vulnerable to attackers who could upload files harboring malware or malicious code. These files have the potential to jeopardize the server's integrity or compromise other clients when downloaded.
- **File Size Exceedances:** Attackers might exploit the system by uploading excessively large files. This could lead to the consumption of server resources, potentially degrading server performance and responsiveness.
- **File Type Violations:** Security concerns might arise if an attacker attempts to upload files with extensions that contravene the server's predefined restrictions. Such files could include executables or scripts, which might be used as tools for compromising the server's security.

These considerations underscore the importance of implementing robust security measures when incorporating file upload functionality into SimpleWebServer.

b) To counteract these potential threats, several security measures could be implemented:

- **Malware Scanning:** The server could incorporate a malware scanning mechanism to inspect uploaded files for any signs of malware or malicious code. This inspection would occur prior to allowing the files to be stored on the server or made available for download by other clients.
- **File Size Management:** Implementing size limitations for uploaded files is a prudent approach. By doing so, the server can prevent excessively large files from consuming precious server resources, preserving overall performance.
- **File Type Controls:** The server could exert control over the types of files accepted for upload by scrutinizing file extensions or content types. Any files that fail to meet the specified criteria could be rejected. Additionally, the server could engage in content validation of uploaded files to ensure they do not contain any potentially harmful code or scripts.

These security mechanisms collectively bolster the server's resilience against the aforementioned threats, promoting a safer environment for file uploads within SimpleWebServer.

A7) In this implementation, we follow a specific sequence of checks and actions when serving files. First, we verify the file's existence and ensure it is not a directory. Then, we assess whether the file size exceeds the defined maximum limit (`MAX_FILE_SIZE`). If the file size surpasses this limit, we log an error entry in the `error_log` file and send a "403 Forbidden" HTTP response code to the client. Otherwise, if the file is within the size limit, we proceed to stream it to the client, as we did previously.

a) If an attacker attempts to download a file like `/dev/random` after the `serveFile()` method has been modified to enforce a maximum file size limit, the server will inspect the file size of `/dev/random` and compare it to the specified maximum size. Given that `/dev/random` is a special file in Unix-like operating systems primarily serving as an interface to the kernel's random number generator, it is unlikely to have a file size exceeding the maximum limit. Consequently, the server will most probably respond with a "403 Forbidden" HTTP status code and record an error entry in the `error_log` file, indicating that the file size constraint has been breached.

b) There exist several alternative methods to implement the maximum file size limitation within the `serveFile()` method. One approach is to utilize the `java.nio` package, enabling the reading of the file in chunks while limiting the number of bytes read to the predefined maximum size. Another option involves employing a servlet container or web framework that offers built-in support for managing file size restrictions, such as Apache Tomcat or Spring Boot. Additionally, you could consider employing third-party libraries like Apache Commons IO, which provide functionality for handling file size limitations. These alternatives offer flexibility in implementing and customizing file size checks according to your application's specific requirements.

...

```
public void serveFile(Socket client, String filename) {  
    try  
    {  
        File file = new File(filename);  
        if (!file.exists() || file.isDirectory()) {  
            client.getOutputStream().write("HTTP/1.0 404 Not Found\n\n".getBytes());  
            return;  
        }  
        if (file.length() > MAX_FILE_SIZE) {  
            logEntry("error_log", "File size limit exceeded for file: " + filename);  
            client.getOutputStream().write("HTTP/1.0 403 Forbidden\n\n".getBytes());  
            return;  
        }  
        FileInputStream fis = new FileInputStream(file);  
        OutputStream os = client.getOutputStream();  
        os.write("HTTP/1.0 200 OK\n\n".getBytes());  
        byte[] buffer = new byte[1024];  
        int bytesRead;  
        while ((bytesRead = fis.read(buffer)) != -1)  
        {  
            os.write(buffer, 0, bytesRead);  
        }  
        fis.close();  
        os.close();  
    }  
    catch (IOException e)  
    {  
        logEntry("error_log", "Error serving file: " + filename);  
    }  
}
```

...